

Technical Report: Cloud-Native E-Commerce on AWS EC2

Course: Cloud Computing and Technologies — Università degli Studi di Milano, A.Y. 2025/2026

Authors: Carlo Invernizzi (65885A), Matteo Bertolotti (65865A)

Live deployment: <https://cloud-computing-uni.duckdns.org>

1. Introduction

This project implements a mini e-commerce application using a microservices architecture deployed on a live AWS EC2 instance. The system is designed around four properties explicitly requested by the course instructor: **resilience**, **security**, **replication**, and **articulated cloud deployment**.

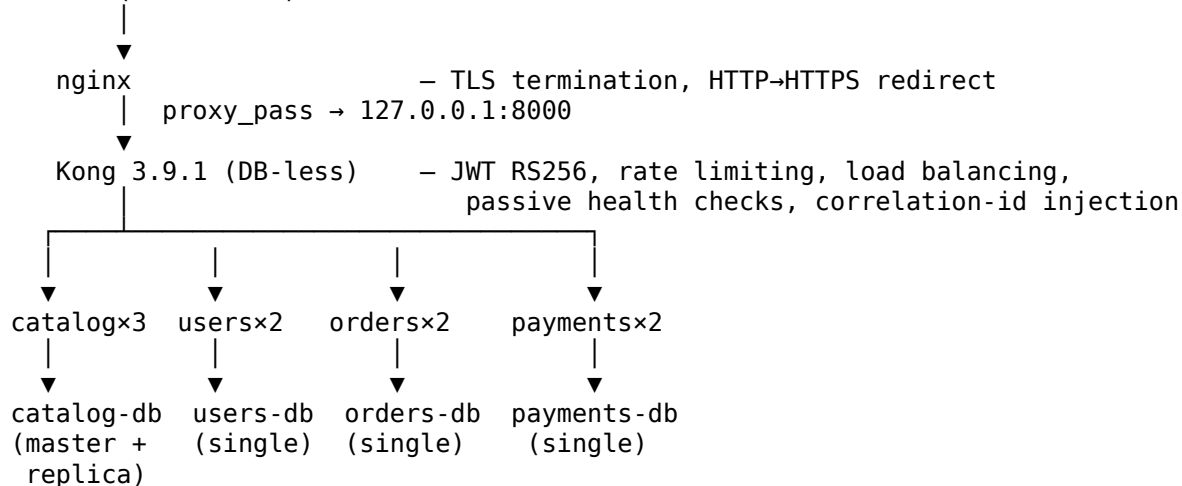
The application exposes a RESTful API through four business microservices — catalog, users, orders, and payments — all reachable exclusively through a Kong API Gateway acting as the single entry point. The gateway handles TLS termination (via nginx), JWT validation, rate limiting, and load balancing. No microservice or database port is accessible from outside the VM.

The technology stack is: Node.js 20 LTS with Fastify 4.28.1, PostgreSQL 16.3, Kong 3.9.1 in DB-less declarative mode, and Prometheus 2.51.2 with Grafana 10.4.2 for observability. All components run as Docker containers orchestrated with Docker Compose on a single Ubuntu 22.04 EC2 instance.

2. Architecture

2.1 Request flow

Client (HTTPS :443)



Prometheus ← scrapes 5 targets every 15s

Grafana ← queries Prometheus

2.2 Microservice boundaries

Each microservice is a Fastify application backed by its own PostgreSQL 16 database (database-per-service pattern). Each service owns its data exclusively and exposes it only through its API. The four services cover distinct bounded contexts:

- **catalog:** product catalog, category taxonomy, stock levels. Exposes public read endpoints and admin write endpoints. Read queries are served from the PostgreSQL read-replica; writes go to the master.
- **users:** registration, authentication, JWT issuance (RS256 private key), profile and address management.
- **orders:** owns the entire purchasing flow — shopping cart, checkout, and order lifecycle. Cart and order entities share one database because they are part of the same bounded context (the customer's purchasing journey).
- **payments:** simulated payment processing. Exposes status and history endpoints publicly via Kong; the internal processing endpoint (/internal/payments/process) is callable only on the Docker network and is intentionally absent from Kong's routing config.

All services are **stateless**: no in-memory sessions, no local file writes. This property is required for safe horizontal replication — any replica can serve any request without coordination. Every service exposes GET /health (liveness) and GET /ready (readiness, checks DB connectivity), as well as GET /metrics in Prometheus format via prom-client.

Cross-service references (e.g., user_id stored in the orders database) are logical foreign keys not enforced at the database level, because they span separate databases. Consistency is maintained through validation calls at API boundaries and compensating transactions (see Section 4.4).

Kong runs in **DB-less declarative mode**: the entire gateway configuration — routes, upstreams, plugins,

JWT consumers — lives in a single version-controlled `kong.yml`, reloadable at runtime via `POST /config` without restarting the container.

Service	Port	Replicas	Database
catalog	3001	3	catalog-db (master + streaming replica)
users	3002	2	users-db (single node)
orders	3003	2	orders-db (single node)
payments	3004	2	payments-db (single node)

2.3 Data model conventions

All tables use UUID primary keys generated with `gen_random_uuid()`, and carry `created_at` / `updated_at` timestamps. Prices are stored as integer cents (`price_cents` `INTEGER`) to avoid IEEE 754 floating-point rounding; the display layer divides by 100. The orders service snapshots `product_name` and `price_cents` at order time so that later catalog changes do not alter historical records. The shipping address is stored as a JSONB snapshot for the same reason. These are deliberate trade-offs: they give up strict referential integrity in exchange for historical accuracy and cross-database independence.

Stock tracking is split into a dedicated `product_stock` table separate from `products`, allowing concurrent stock updates without locking the product row. Stock is only reserved at checkout — not when the customer adds an item to the cart — to prevent users from hoarding inventory through abandoned sessions. This choice means two users can simultaneously hold the same item in their carts; whoever completes checkout first wins, and the second receives an HTTP 409 with an explicit availability error.

3. Security

Security is implemented as a multi-layer stack (defense in depth): network perimeter, gateway, and application layer.

3.1 JWT RS256 authentication

The users service is the sole JWT issuer. It holds the **RSA private key** (mounted read-only at `JWT_PRIVATE_KEY_PATH`) and signs tokens with RS256. Access tokens expire after 900 seconds (15 minutes); refresh tokens after 7 days. The token payload carries `sub` (user UUID), `email`, `role`, `iat`, and `exp`.

Kong holds only the **RSA public key**, embedded inline in `kong.yml` under `jwt_secrets` with `algorithm: RS256`. Kong verifies the signature and the `exp` claim on every protected route before forwarding the request. Microservices that need to check identity (catalog write endpoints, orders, payments) also mount the public key via a read-only Docker volume and re-validate the token at the application layer. This two-checkpoint approach ensures that even if a request somehow bypasses Kong on the internal network, the service will still reject an invalid token.

users (private.pem) → sign token

Kong (public.pem) → verify signature + exp [checkpoint 1]

service (public.pem) → verify signature + exp [checkpoint 2]

The Kong consumer is keyed on the `iss` claim ("users-service"), which Kong uses to look up the correct public key when multiple issuers could exist.

3.2 Route-level access control

Kong's route configuration enforces differentiated access policies using longest-prefix matching:

Route pattern	Authentication	Rate limit (per IP)
/api/v1/users/auth/login	None	10 req/min
/api/v1/users/auth/register	None	10 req/min
/api/v1/users/auth/* (refresh, logout)	None	100 req/min
/api/v1/catalog/* (GET/HEAD)	None	100 req/min
/api/v1/catalog/products (POST/PUT/PATCH/DELETE)	JWT RS256	100 req/min
/api/v1/orders/*, /api/v1/cart/*	JWT RS256	100 req/min
/api/v1/payments/*	JWT RS256	10 req/min

CORS preflight requests (`OPTIONS`) bypass JWT validation globally (`run_on_preflight: false`) to allow browser-initiated requests without credentials. The `correlation-id` global plugin injects an `X-Correlation-ID` UUID on every request if the client did not supply one, enabling end-to-end tracing across service logs.

3.3 Network isolation

The AWS Security Group allows only ports 22 (SSH, restricted to developer IP addresses), 80 (HTTP, redirected to HTTPS by nginx), and 443 (HTTPS). Kong's Admin API (8001), all four service ports (3001-3004), all four database ports (5432), Prometheus (9090), and Grafana (3000) are accessible only from localhost or the backend Docker bridge network — they are never reachable from the internet.

3.4 Password storage and secrets management

Passwords are hashed with bcrypt at cost factor 12 and stored in a VARCHAR(60) column. The plain-text password is never stored, never logged, and cleared from memory immediately after hashing. All secrets (database passwords, JWT key paths, admin credentials) are injected via environment variables at runtime. The .env file is gitignored; .env.example contains only placeholder values. Secrets (RSA private key, DB passwords) are gitignored and injected via environment variables; only the public key is committed to the repository.

3.5 Input validation

Every Fastify route has a JSON Schema defined in its `schema.body`, `schema.querystring`, and `schema.params` properties. Fastify compiles these schemas with AJV and rejects malformed requests with a 400 error before they reach the handler, preventing injection and unexpected input at the application boundary.

3.6 Structured audit logging

Every service uses pino for structured JSON logging. All log entries include the `correlationId` header injected by Kong's `correlation-id` plugin, enabling end-to-end request tracing across service boundaries. Security-relevant events — login attempts (successful and failed), token issuance, order modifications, payment state transitions — are logged at the `info` or `warn` level with the user's UUID and role. The plaintext password, JWT payload, and database credentials are never present in log output. In production the logs are consumed by Docker's `json-file` driver and can be forwarded to an external aggregator; this is out of scope for the current deployment but the structured format is designed to be ingestible by tools such as Loki or CloudWatch Logs without transformation.

4. Resilience and Replication

4.1 Service replication

Stateless microservices are deployed in multiple instances (`catalog` ×3, `users/orders/payments` ×2). Docker DNS resolves each service name to all instance IPs; Kong distributes incoming requests in round-robin across them.

Replica counts are declared in `docker-compose.yml` via `deploy.replicas`. No `container_name` or `host-port` binding is set on replicated services — both are incompatible with scaling. Docker's internal DNS resolves the service hostname (e.g., `catalog`) to all live container IPs; Kong's round-robin upstream load-balances across them. When a node is killed, Docker DNS removes its IP from the resolution pool immediately, so Kong stops routing to it. The passive health checker provides a secondary defense layer for 'zombie' instances that remain reachable but return 5xx errors.

The `catalog` service has 3 replicas because it handles the highest read traffic (product browsing). All other services have 2 replicas, providing fault tolerance without excessive resource consumption on a `t3.medium` instance.

4.2 Health checks and passive failover

Every container declares a Docker healthcheck that polls `/health` with `wget`. Startup order is enforced via `depends_on`: `condition: service_healthy` so that a service does not start until its database is accepting connections. All services have `restart: unless-stopped`.

Kong upstreams use **passive health checks**: after 3 consecutive HTTP 5xx responses or timeouts from a target, Kong marks it unhealthy and stops routing traffic to it; after 2 consecutive successes it is readmitted. Active polling is disabled — passive checks introduce zero overhead and are sufficient for the replica counts deployed.

```
passive:
  unhealthy:
    http_failures: 3
    http_statuses: [500, 502, 503, 504]
    timeouts: 3
  healthy:
    successes: 2
```

All Kong service definitions set 3-second timeouts on `connect_timeout`, `write_timeout`, and `read_timeout`, bounding worst-case latency for inter-service calls. Services handle `SIGTERM` for graceful shutdown: the Fastify server stops accepting new connections and waits for in-flight requests to complete before the process exits, preventing incomplete responses during rolling restarts.

4.3 PostgreSQL streaming replication

The `catalog` database runs as a master + hot standby pair:

- **catalog-db-master**: PostgreSQL with `wal_level=replica`, `max_wal_senders=4`, `hot_standby=on`.
- **catalog-db-replica**: bootstrapped via `pg_basebackup -R` on first start, which writes `standby.signal`

and `primary_conninfo` automatically. Runs with `hot_standby=on`.

The catalog service receives two connection strings via environment variables: `DATABASE_URL` (master) for writes and stock reservations, and `DATABASE_REPLICA_URL` (replica) for all public read queries (product listings, search, detail pages). Separating reads to the replica reduces master load and improves read latency.

The other three databases are single-node. Streaming replication adds operational complexity — failover promotion, replication lag monitoring, split-brain prevention — that is not justified at this scale for services that are not read-heavy.

4.4 Saga pattern at checkout

The checkout operation (`POST /api/v1/cart/checkout`) is the only cross-service write flow. It implements a synchronous Saga with compensating transactions:

1. **orders** validates cart contents, calls **catalog** to verify product availability.
2. **catalog** atomically reserves stock (`quantity_available--`, `quantity_reserved++`).
3. **orders** creates the order record and calls **payments** to process payment.
4. On payment success: stock reservation is consumed (`quantity_reserved--`).
5. On payment failure: the orders service issues a compensation call to `/internal/catalog/stock/release` to restore the reserved units; the order is then marked as cancelled but the stock count returns to its pre-checkout value.

Stock is not reserved at cart-add time, only at checkout. This avoids phantom out-of-stock situations caused by abandoned carts, at the cost of a potential last-second conflict — handled gracefully with an HTTP 409 response and a clear error message.

The payments service is intentionally simulated: it introduces a 200–500 ms artificial delay to mimic real provider latency and uses deterministic rules for failure (a credit card number ending in 0000 always fails). This makes the compensation branch reproducible during exams without depending on external services or network conditions. The simulation logic is self-contained in the payments service; replacing it with a real provider adapter is an isolated code change that does not affect any other service or the gateway configuration.

5. Cloud Deployment and Observability

5.1 Infrastructure

The entire system runs on a single AWS EC2 instance (Ubuntu 22.04 LTS, t3.medium/t3.large, eu-north-1 region). All components are Docker containers managed by Docker Compose. Named volumes persist database data and Prometheus metrics across container restarts; `docker compose down -v` is required to purge them.

The EC2 instance is stopped between sessions to remain within free-tier limits. Cold start (volumes already populated) takes approximately 60 seconds for all health checks to pass.

Database data and Prometheus time-series are stored in Docker named volumes (`catalog-db-master-data`, `catalog-db-replica-data`, `users-db-data`, `orders-db-data`, `payments-db-data`, `prometheus-data`, `grafana-data`). A `docker compose down` preserves the volumes; `docker compose down -v` destroys them. In the current setup the EC2 instance's EBS root volume acts as the persistent storage backing these volumes, so data survives VM restarts. Full database backup is not automated for this academic deployment but the volume structure is straightforward to snapshot with standard EBS snapshot tooling.

5.2 HTTPS and DNS

A free DuckDNS subdomain (`cloud-computing-uni.duckdns.org`) points to the EC2 public IP. HTTPS is handled by a host-level nginx instance acting as a TLS-terminating reverse proxy:

```
Client :443 → nginx (TLS) → proxy_pass 127.0.0.1:8000 → Kong
```

```
Client :80 → nginx 301 redirect → https://...
```

The TLS certificate is issued by Let's Encrypt via `certbot --nginx`. The provisioning script (`deployment/aws/setup-https.sh`) is idempotent: it installs nginx and certbot, writes the virtual-host config, obtains or renews the certificate, and registers a daily auto-renewal cron (`0 3 * * * certbot renew --quiet && systemctl reload nginx`). Re-running it on an already-provisioned VM is safe.

5.3 Observability

Prometheus scrapes **five targets** every 15 seconds: Kong (via the prometheus gateway plugin, which exports per-route status codes, latency histograms, and upstream health metrics) and the four microservices (each exposing prom-client default metrics plus custom HTTP request histograms). Data is retained for 7 days.

Three alert rules are defined in `alerts.yml`:

Alert	Expression	Threshold	Severity
ServiceDown	<code>up == 0</code>	1 min	critical
HighErrorRate	<code>5xx / total (5m rate)</code>	> 5%	warning
SlowResponse	<code>p99 latency histogram (5m)</code>	> 1s	warning

Alert	Expression	Threshold	Severity
-------	------------	-----------	----------

Grafana auto-provisions the Prometheus datasource and the “Microservices Overview” dashboard, which contains four panels: request rate per service, error rate (5xx percentage), p99 latency per service, and service health. Anonymous viewer access is enabled for exam demo convenience.

5.4 Reproducible demos

Four shell scripts in docs/demos/ drive the oral examination scenarios:

- demo-resilience.sh: kills one catalog replica under k6 load; Kong routes around it with zero down-time.
- demo-security.sh: demonstrates JWT validation rejection and login rate limiting (10 rapid requests → HTTP 429).
- demo-replication.sh: kills the catalog DB master; reads continue from the hot standby.
- demo-checkout.sh: executes a full checkout Saga including a simulated payment failure and the resulting stock compensation.

Four reproducible demo scripts in docs/demos/ cover resilience, security, replication and checkout scenarios end-to-end.

6. Design Decisions and Limitations

6.1 Kong as a single container

Kong runs as a single container (container_name: kong) in this deployment. The fixed container name and the single-port binding (8000:8000) are incompatible with docker compose scale. In a production setup, a second Kong instance would be placed behind an AWS ELB or an nginx upstream, which is out of scope for a single-VM academic project. This is not an architectural constraint: Kong is stateless in DB-less mode, so scaling it is a deployment-layer change that does not require code changes.

6.2 No automated CI/CD

Deployment is manual (SSH + docker compose up -d). A GitHub Actions pipeline was evaluated but set aside because the EC2 instance is stopped between sessions — a webhook-triggered deploy would silently fail when the VM is off. The provisioning and HTTPS scripts are idempotent and documented, so a fresh deployment from scratch takes fewer than ten minutes.

6.3 Free-tier constraints and scope

Several architectural choices reflect the AWS free tier and the academic scope of the project:

- Single EC2 VM instead of multiple instances; Docker Compose instead of ECS or Kubernetes.
- In-container Prometheus and Grafana instead of CloudWatch.
- DuckDNS free subdomain instead of Route 53.
- Simulated payment processing instead of a real payment provider integration.
- No frontend — the API is consumed directly (tested via Bruno during development, k6 for load tests).

These constraints are acknowledged explicitly and align with the course requirement to demonstrate cloud computing concepts on a real deployment, not to build a production-grade system.

6.4 What would change in production

For a production deployment the main changes would be: (a) replace the single Kong container with two instances behind an AWS ELB, removing the gateway as a single point of failure; (b) replace Docker Compose with ECS or Kubernetes to enable auto-scaling and rolling updates without SSH access; (c) move databases to RDS PostgreSQL with Multi-AZ standby, offloading replication management; (d) replace the DuckDNS subdomain with a Route 53-managed domain and an ACM certificate on the ELB; (e) add a centralized log aggregation pipeline (Loki or CloudWatch) and distributed tracing. Each of these changes is incremental and independent — the service code, the API contracts, and the database schemas remain unchanged.